

T.R.
GEBZE TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

**ASL GLOVE: A WEARABLE AMERICAN SIGN
LANGUAGE TRANSLATOR USING INERTIAL
SENSING AND ON-DEVICE DEEP LEARNING**

BERK HAKAN ÖGE - UTKU GÖKÇEK

**SUPERVISOR
DR. ÖĞR. ÜYESİ ALP ARSLAN BAYRAKÇI**

**GEBZE
2026**

T.R.
GEBZE TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

**ASL GLOVE: A WEARABLE AMERICAN
SIGN LANGUAGE TRANSLATOR USING
INERTIAL SENSING AND ON-DEVICE
DEEP LEARNING**

BERK HAKAN ÖGE - UTKU GÖKÇEK

SUPERVISOR
DR. ÖĞR. ÜYESİ ALP ARSLAN BAYRAKÇI

2026
GEBZE

	GRADUATION PROJECT JURY APPROVAL FORM
---	--

This study has been accepted as an Undergraduate Graduation Project in the Department of Computer Engineering on 18/06/2026 by the following jury.

JURY

Member

(Supervisor) : Dr. Öğr. Üyesi Alp Arslan BAYRAKÇI

Member : Dr. Öğr. Üyesi Alp Arslan BAYRAKÇI

Member : Prof. Dr. Erkan ZERGEROĞLU

ABSTRACT

Sign language is the primary means of communication for millions of deaf and hard-of-hearing people, yet the scarcity of interpreters in everyday settings creates a persistent barrier with the hearing world. This project presents a self-contained, wearable glove that recognises American Sign Language (ASL) gestures and translates them into spoken output in real time, without a camera or any external infrastructure.

The system places a compact inertial measurement unit (MPU6050) and a microcontroller (Seeed Studio XIAO ESP32-C3) on the nail of each of the five fingers. Four “slave” modules stream fused orientation and angular-rate data over the ESP-NOW protocol to a fifth “master” module on the middle finger, which aggregates a 45-channel feature vector at 25 Hz and forwards it to a mobile phone over Bluetooth Low Energy. A Flutter application captures a fixed-length window of sensor data on demand, runs a quantised one-dimensional convolutional neural network entirely on-device, and speaks the recognised letter or word through text-to-speech.

A central finding of the work is the decisive role of the feature representation. Static handshape recognition based on absolute finger orientation proved fragile, because the inertial reference frame cannot be reproduced exactly between sessions and because the limited angular resolution of the sensors cannot separate handshapes that differ only by small thumb displacements. The project therefore adopts distinctive per-letter *motions* together with *absolute* (non-orientation-normalised) features, relocating the discriminative information into the calibration-independent angular-rate channels. The deployed models occupy 137 KB of flash, require 22.7 KB of RAM, and execute in approximately 1 ms per inference; the wireless link delivers all five finger streams at 25 Hz with negligible packet loss. The project demonstrates that accurate, low-latency, fully on-device sign-language translation is achievable on inexpensive commodity hardware, and it documents the engineering trade-offs—radio coexistence, power integrity, thermal behaviour, and feature invariance—that govern such a system.

Keywords: sign language recognition, wearable computing, inertial measurement unit, ESP-NOW, Bluetooth Low Energy, edge machine learning, one-dimensional convolutional neural network, gesture recognition.

ÖZET

İşaret dili, milyonlarca işitme engelli birey için temel iletişim aracıdır; ancak günlük hayatta tercüman bulunmaması, işiten dünya ile aralarında sürekli bir iletişim engeli oluşturmaktadır. Bu proje, Amerikan İşaret Dili (ASL) hareketlerini tanıyıp gerçek zamanlı olarak sesli çıktıya dönüştüren, kamera veya herhangi bir dış altyapı gerektirmeyen, kendi kendine yeten giyilebilir bir eldiven sunmaktadır.

Sistem, beş parmağın her birinin tırnağına kompakt bir eylemsizlik ölçüm birimi (MPU6050) ve bir mikrodenetleyici (Seeed Studio XIAO ESP32-C3) yerleştirir. Dört “uydu” modül, füzyonlanmış yönelim ve açısız hız verilerini ESP-NOW protokolü üzerinden orta parmağındaki beşinci “ana” modüle iletir. Ana modül, 25 Hz hızında 45 kanallı bir öznitelik vektörü oluşturarak bunu Bluetooth Düşük Enerji (BLE) üzerinden bir akıllı telefona aktarır. Bir Flutter uygulaması, istek üzerine sabit uzunlukta bir sensör penceresi yakalar, nicelenmiş tek boyutlu bir evrişimsel sinir ağını tamamen cihaz üzerinde çalıştırır ve tanınan harf veya kelimeyi metinden konuşmaya çevirme ile seslendirir.

Çalışmanın temel bulgularından biri, öznitelik temsiliinin belirleyici önemidir. Mutlak parmak yönelimine dayalı statik el şekli tanıma, eylemsizlik referans çerçevesinin oturumlar arasında tam olarak yeniden üretilmemesi ve sensörlerin sınırlı açısız çözünürlüğünün yalnızca küçük başparmak hareketleriyle ayrılan el şekillerini ayırt edememesi nedeniyle kırılmalı kalmıştır. Bu nedenle proje, ayırt edici bilgiyi kalibrasyondan bağımsız açısız hız kanallarına taşıyan, harfe özgü belirgin *hareketler* ile *mutlak* öznitelikleri benimsemiştir. Dağıtılan modeller 137 KB flash ve 22.7 KB RAM kullanır ve çıkarım başına yaklaşık 1 ms sürede çalışır; kablosuz bağlantı beş parmağın tüm akışlarını 25 Hz hızında ihmal edilebilir paket kaybıyla iletir. Proje, düşük maliyetli, yaygın donanım üzerinde doğru, düşük gecikmeli ve tamamen cihaz içi işaret dili çevirisinin uygulanabilir olduğunu göstermekte ve böyle bir sistemi yöneten mühendislik ödünleşimlerini—radyo birlikte çalışabilirliği, güç bütünlüğü, termal davranış ve öznitelik değişmezliği—belgelemektedir.

Anahtar Kelimeler: işaret dili tanıma, giyilebilir teknoloji, eylemsizlik ölçüm birimi, ESP-NOW, Bluetooth Düşük Enerji, uç birim makine öğrenmesi, tek boyutlu evrişimsel sinir ağı, hareket tanıma.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor for the guidance, patience, and constructive feedback offered throughout this project. We also thank the Department of Computer Engineering at Gebze Technical University for providing the environment and resources that made this work possible. Finally, we are deeply grateful to our families and friends for their continuous support and encouragement during our studies.

Berk Hakan Öge

Utku Gökçek

LIST OF SYMBOLS AND ABBREVIATIONS

Symbol or

Abbreviation : Explanation

ASL	: American Sign Language
IMU	: Inertial Measurement Unit
MCU	: Microcontroller Unit
BLE	: Bluetooth Low Energy
GATT	: Generic Attribute Profile (BLE)
MTU	: Maximum Transmission Unit
CNN	: Convolutional Neural Network
1-D CNN	: One-Dimensional Convolutional Neural Network
DOF	: Degrees of Freedom
DLPF	: Digital Low-Pass Filter
DSP	: Digital Signal Processing
EI	: Edge Impulse (model development platform)
TFLite	: TensorFlow Lite
int8	: 8-bit integer (quantised) representation
TTS	: Text-to-Speech
USB	: Universal Serial Bus
CDC	: Communications Device Class (USB serial)
RF	: Radio Frequency
LDO	: Low-Dropout (voltage regulator)
BOD	: Brown-Out Detector
RMS	: Root Mean Square
f_s	: Sampling frequency (25 Hz)
P, R, Y	: Pitch, Roll, Yaw (Euler angles)
g_x, g_y, g_z	: Angular rate about the x, y, z axes
a_x, a_y, a_z	: Linear acceleration along the x, y, z axes

CONTENTS

Abstract	iv
Özet	v
Acknowledgement	vi
List of Symbols and Abbreviations	vii
Contents	x
List of Figures	xi
List of Tables	xii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Objectives and Scope	2
1.4 Contributions	2
1.5 Thesis Outline	3
2 Background and Related Work	4
2.1 Sign Language and Fingerspelling	4
2.2 Approaches to Sign Language Recognition	4
2.3 Inertial Measurement Units and Sensor Fusion	5
2.4 Wireless Protocols for Wearables	5
2.5 Machine Learning at the Edge	5
3 System Overview and Hardware Architecture	7
3.1 Requirements	7
3.2 Component Selection	7
3.3 The Finger Module	7
3.4 Network Topology: Master and Slaves	8
3.5 Evolution of the Design	10
3.6 Power	11

4	Firmware and Sensor Signal Processing	12
4.1	Per-Finger Signal Processing	12
4.1.1	Acquisition and Filtering	12
4.1.2	Orientation Estimation	12
4.1.3	Calibration and Bias Tracking	12
4.1.4	Packet Format	13
4.2	The Slave Sketch	13
4.3	The Master Sketch	13
4.3.1	Aggregation and Channel Order	14
4.3.2	Radio Coexistence and the BLE MTU	14
5	Data Acquisition, Feature Engineering, and Modelling	15
5.1	Data Acquisition	15
5.2	Signal Window and Channels	15
5.3	Feature Representation	15
5.3.1	The Problem with Absolute Orientation for Static Handshapes	15
5.3.2	Relative (Orientation-Invariant) Features	16
5.3.3	Dynamic Gestures with Absolute Features	16
5.4	Augmentation and Dataset Splitting	17
5.5	Network Architecture	17
5.6	Quantisation and Deployment	17
6	Mobile Application	18
6.1	Architecture	18
6.2	Bluetooth Link and Packet Parsing	18
6.3	Gated Capture	18
6.4	On-Device Inference Pipeline	19
6.5	Per-Model Configuration	19
6.6	Output	19
7	Results and Evaluation	21
7.1	Wireless Link Reliability	21
7.2	On-Device Model Performance	21
7.3	Recognition Accuracy	22
7.3.1	The Single-Session Caveat	24
7.3.2	Live Findings and the Role of Feature Representation	24
7.4	Power and Thermal Behaviour	24
8	Conclusion and Future Work	26
8.1	Summary	26

8.2	Limitations	26
8.3	Future Work	27
8.4	Concluding Remarks	27
Bibliography		28
Appendices		29

LIST OF FIGURES

3.1	A nail-mounted finger module: XIAO ESP32-C3 stacked over an MPU6050 with an isolating pad between them.	8
3.2	The assembled glove. Five finger modules (thumb, index, ring, and pinky as slaves; middle as master) are mounted over the nails and wired to a single power rail.	9
3.3	On-hand data flow. Four slave modules each fuse their own IMU and broadcast a packet over ESP-NOW at 25 Hz; the middle-finger master adds its own IMU, assembles the 45-channel feature vector, and streams it to the phone over BLE, where the quantised CNN classifies it and text-to-speech speaks the result.	10
6.1	The mobile application. Left: Words mode, which speaks a recognised sign immediately. Centre: Letters mode, which accumulates finger-spelled letters into a buffer with Speak and Clear controls. Right: the Settings screen, exposing the confidence threshold, text-to-speech voice, and the Letters-mode auto-clear interval.	20
7.1	Validation-set confusion matrices from Edge Impulse. Top: the 26-letter model. Bottom: the 19-word model. Both are fully diagonal—every class is recognised with no within-session confusion—so the residual errors of interest are the cross-session ones discussed in the live findings below, not anything visible here.	23

LIST OF TABLES

3.1	Network roles and data paths.	9
7.1	Representative five-second link diagnostics from the master (BLE connected).	21
7.2	On-device characteristics of the quantised model (Edge Impulse estimate, Raspberry Pi 4 class target).	22
7.3	Edge Impulse validation-set accuracy of the main model variants. All figures are within-session (the data is recorded in a single session and split internally); they measure separability under fixed calibration, not cross-session generalisation—see the caveat below.	22

1. INTRODUCTION

1.1. Motivation

According to the World Health Organization, more than 5% of the world’s population lives with disabling hearing loss. For many of these people, a signed language—rather than a written or spoken one—is their first and most natural means of communication. American Sign Language (ASL) is a complete, grammatically rich language used by a large community across North America and beyond. Despite this, communication between signers and the hearing majority remains difficult in everyday situations such as shops, clinics, and public offices, because qualified interpreters are scarce and cannot be present on demand.

Technology offers a path to narrow this gap. If a wearable device could recognise signed gestures and render them as speech, a signer could be understood by a hearing person who knows no sign language, with no third party required. Such a device must be *portable, low-latency, affordable*, and ideally *self-contained*, so that it works anywhere without a network connection or a fixed camera installation. These constraints motivate the design presented in this thesis: a glove that senses hand and finger motion with small inertial sensors and performs the entire recognition pipeline on-device.

1.2. Problem Statement

The core problem is to translate a vocabulary of ASL gestures into text and speech using only wearable inertial sensing, subject to the practical constraints of a battery- or USB-powered wearable. Concretely, the system must:

- sense the configuration and motion of the five fingers with sufficient fidelity to distinguish a useful vocabulary of letters and words;
- transmit that data wirelessly and reliably from the hand to a processing device;
- classify gestures accurately and with low latency;
- operate without a camera, a cloud service, or a wired tether during use; and
- remain robust to the practical realities of a wearable—imperfect re-calibration between uses, limited power, and the heat generated by the on-hand electronics.

1.3. Objectives and Scope

The objectives of this project are:

1. To design and build a five-finger sensing glove based on commodity inertial measurement units and microcontrollers.
2. To establish a reliable wireless data path from the glove to a mobile phone.
3. To collect a labelled dataset of ASL gestures and to train a compact neural network suitable for on-device inference.
4. To develop a mobile application that performs inference and produces spoken output.
5. To evaluate the accuracy, latency, and practical robustness of the complete system, and to document the engineering decisions and trade-offs encountered.

The scope is limited to a single hand, a fixed vocabulary of fingerspelling letters and a set of common one-handed word signs, and recognition of deliberate, segmented gestures (one gesture captured per request) rather than continuous, unsegmented signing.

1.4. Contributions

The principal contributions of this work are:

- A **no-hub, master/slave wearable architecture** in which one finger module aggregates the data of the other four and bridges directly to a phone, eliminating the separate wrist-mounted receiver and the external antennas of earlier iterations.
- An empirical study of **feature representation** for inertial sign recognition, showing why orientation-normalised (“relative”) features and absolute features each succeed or fail depending on whether a gesture is static or dynamic, and arriving at a *dynamic-gesture, absolute-feature* formulation that is robust to imperfect re-calibration.
- A **train/inference-parity pipeline** in which the exact windowing, feature transform, and normalisation used during data collection are reproduced on the phone, together with a quantised model that runs in roughly one millisecond on-device.
- A documented account of the **system-level engineering trade-offs**—radio coexistence, USB throughput, power integrity, and thermal behaviour—that are rarely reported but decisive in practice.

1.5. Thesis Outline

Chapter 2 reviews sign-language recognition approaches, inertial sensing, and edge machine learning. Chapter 3 describes the system requirements and the hardware architecture, including the evolution of the design. Chapter 4 details the firmware and the sensor signal-processing chain. Chapter 5 presents the data-acquisition pipeline, the feature-representation study, and the neural network. Chapter 6 describes the mobile application and the on-device inference path. Chapter 7 reports and discusses the evaluation results. Chapter 8 concludes and outlines future work.

2. BACKGROUND AND RELATED WORK

2.1. Sign Language and Fingerspelling

A signed language conveys meaning through handshape, orientation, location, movement, and non-manual markers such as facial expression. ASL has a dedicated set of one-handed handshapes for the letters of the alphabet, used for *fingerspelling*—spelling out words letter by letter—which is common for names and for words that lack a dedicated sign. Most letters are static handshapes; two, **J** and **Z**, are defined by a traced motion. Word-level signs are typically dynamic and may be one- or two-handed. Because the device in this project instruments a single hand, the vocabulary is restricted to fingerspelling and to one-handed word signs.

A practical consequence for sensing is that several fingerspelling letters differ only in subtle thumb placement against a closed fist—for example **A**, **S**, **T**, **M**, and **N**. The ability of any sensing modality to separate these handshapes is therefore a key indicator of its resolution.

2.2. Approaches to Sign Language Recognition

Sign-language recognition (SLR) systems fall broadly into two families.

Vision-based systems use one or more cameras and computer-vision or deep-learning models to interpret hand and body pose. They require no instrumentation of the signer but depend on controlled lighting, a clear line of sight, and significant computation, and they raise privacy concerns because they continuously capture video.

Sensor-based (glove) systems instrument the hand directly with sensors such as flex sensors, which measure finger bend, or inertial measurement units (IMUs), which measure orientation and motion. Gloves work in any lighting, do not capture the surroundings, and can run on small microcontrollers, at the cost of requiring the user to wear the device. This project adopts the glove approach using IMUs, which—unlike flex sensors—capture both static finger orientation and dynamic motion, and are available as inexpensive, easily interfaced modules.

2.3. Inertial Measurement Units and Sensor Fusion

An IMU such as the InvenSense MPU6050 [1] integrates a three-axis gyroscope and a three-axis accelerometer. The accelerometer measures the gravity vector (giving an absolute tilt reference) plus linear acceleration; the gyroscope measures angular rate. Neither sensor alone yields a stable orientation: accelerometer-only tilt is corrupted by motion, while integrating the gyroscope alone accumulates drift.

Sensor fusion combines the two to obtain a stable estimate. The Kalman filter [2] is the classical optimal estimator for this purpose and is widely used to fuse accelerometer and gyroscope measurements into pitch and roll angles. A practical complication is that the gyroscope’s zero-rate bias drifts with temperature; this is commonly mitigated by continuously re-estimating the bias whenever the device is detected to be still. Both techniques are used in this project’s firmware (Chapter 4).

A fundamental limitation of a six-axis IMU is that yaw (rotation about the gravity axis) cannot be referenced absolutely without a magnetometer, and that small static orientation differences are near the noise floor of a low-cost part. These limitations directly shape the feature-representation findings of Chapter 5.

2.4. Wireless Protocols for Wearables

Body-worn sensor networks require a short-range, low-power link. Two relevant technologies are used here. **ESP-NOW** [3] is a connectionless, low-latency protocol built on the Wi-Fi physical layer by Espressif, well suited to many-to-one streaming between ESP32-family devices without the overhead of a full Wi-Fi association. **Bluetooth Low Energy (BLE)** is the standard for communicating with smartphones; its GATT profile exposes *characteristics* that a phone can subscribe to for streaming notifications. The system in this thesis uses ESP-NOW for finger-to-finger communication and BLE for the final hop to the phone (Chapter 3).

2.5. Machine Learning at the Edge

Running inference on a constrained device (“edge” or “TinyML”) requires small models and efficient execution. Two ideas are central. First, **one-dimensional convolutional neural networks (1-D CNNs)** are an effective and lightweight architecture for time-series and human-activity recognition [4], [5], learning local temporal patterns with far fewer parameters than recurrent models. Second, **quantisation** converts a trained 32-bit floating-point model to 8-bit integers, shrinking it roughly four-fold

and accelerating inference, usually with negligible accuracy loss; TensorFlow Lite [6] provides this conversion and a portable runtime. The models in this project are 1-D CNNs trained with the Edge Impulse platform [7] and deployed as int8 TensorFlow Lite files inside a mobile application.

3. SYSTEM OVERVIEW AND HARDWARE ARCHITECTURE

3.1. Requirements

From the objectives in Chapter 1, the hardware must (i) sense all five fingers, (ii) transmit their data to a phone without a wired tether, (iii) be small and light enough to wear, and (iv) be buildable from affordable, available parts. These requirements drove the choice of one IMU per finger, an ESP32-class microcontroller per finger for local processing and radio, and a phone as the inference and output device.

3.2. Component Selection

Each finger module pairs a **Seeed Studio XIAO ESP32-C3** [8] with an **InvenSense MPU6050** [1]. The XIAO ESP32-C3 was chosen for its very small footprint, its integrated Wi-Fi/BLE radio (enabling both ESP-NOW and BLE), its USB-C interface for flashing and power, and its low cost. The MPU6050 provides the six axes of inertial data over I²C and is likewise inexpensive and well documented. The phone runs a cross-platform Flutter application, so no platform-specific hardware is required on the output side.

3.3. The Finger Module

A finger module is a compact stack: the XIAO and the MPU6050 are soldered together with very short wires and separated by an insulating pad that also limits heat transfer from the microcontroller to the sensor; a second heat-spreading pad sits on top of the microcontroller and is left exposed to open air so it can actually dissipate heat rather than trap it. The complete stack mounts on the back of the fingertip (over the nail). Mounting on the nail—rather than along the finger segments—keeps the sensor on a rigid part of the finger, simplifies the wiring, and avoids fitting electronics into the knuckle gaps. Figure 3.1 shows a single assembled module.

The I²C bus uses pins D4 (GPIO6, SDA) and D5 (GPIO7, SCL) on the XIAO at 400 kHz. Each MPU6050 is configured with a digital low-pass filter at 10 Hz, a gyroscope full-scale range of ± 500 °/s, and an accelerometer range of ± 4 g.

Five such modules make up the complete device. Figure 3.2 shows the glove

as worn: one module over each fingernail, with the middle-finger module acting as the master and the remaining four as slaves. All five share a common power rail, and only the master carries the Bluetooth link to the phone, so the wiring the user sees is deliberately minimal—short inter-module runs and a single cable to the power source.



Figure 3.1: A nail-mounted finger module: XIAO ESP32-C3 stacked over an MPU6050 with an isolating pad between them.

3.4. Network Topology: Master and Slaves

The five modules form a star network on the hand. The **middle-finger module is the master**; the **thumb, index, ring, and pinky modules are slaves**. Each slave reads and fuses its own IMU and transmits a small packet to the master over ESP-NOW at 25 Hz. The master reads its own IMU, collects the latest packet from each slave, and assembles a single 45-channel feature vector (five fingers $\times$ nine fields), which it forwards to the phone over BLE. This topology is illustrated in Figure 3.3 and summarised in Table 3.1.



Figure 3.2: The assembled glove. Five finger modules (thumb, index, ring, and pinky as slaves; middle as master) are mounted over the nails and wired to a single power rail.

Table 3.1: Network roles and data paths.

Module (finger)	Role	Outgoing link
Thumb (1)	Slave	ESP-NOW → master
Index (2)	Slave	ESP-NOW → master
Middle (3)	Master	BLE → phone
Ring (4)	Slave	ESP-NOW → master
Pinky (5)	Slave	ESP-NOW → master

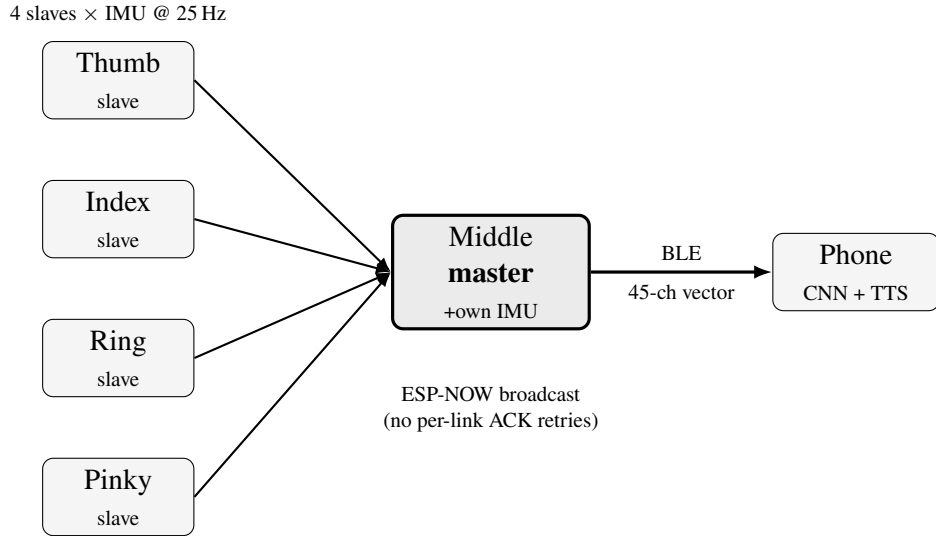


Figure 3.3: On-hand data flow. Four slave modules each fuse their own IMU and broadcast a packet over ESP-NOW at 25 Hz; the middle-finger master adds its own IMU, assembles the 45-channel feature vector, and streams it to the phone over BLE, where the quantised CNN classifies it and text-to-speech speaks the result.

3.5. Evolution of the Design

The final architecture was reached iteratively, and the intermediate designs are instructive.

Iteration 1 — wrist hub with external antennas. The first design used a separate ESP32 development board worn at the wrist as a receiver, and clipped an external U.FL antenna to each finger module to improve the radio link to the wrist. Measurements revealed three problems: the wrist-to-fingertip distance made the link sensitive to hand pose (a closed fist shadowed the inner fingers); the five external antennas, bunched together when the hand closed, coupled and interfered; and routing the antenna coax along moving fingers was mechanically fragile.

Iteration 2 — nail-mounted modules, no antennas, no hub. Relocating the receiver onto the middle fingertip reduced the worst-case link distance from roughly 20 cm (wrist) to a few centimetres (adjacent fingertip), which made the on-chip antennas more than sufficient and removed the external antennas and their coupling entirely. Because the master fingertip can talk to the phone directly over BLE, the separate wrist hub was also eliminated. This single change resolved the antenna-coupling, hand-shadowing, and mechanical-fragility problems together, and reduced the part count.

Sampling rate. The data rate was reduced from an initial 50 Hz to 25 Hz. At 50 Hz the master could not reliably both receive four ESP-NOW streams and emit BLE notifications on a single radio; 25 Hz leaves ample headroom, and human hand motion

is well below the resulting 12.5 Hz Nyquist limit, so no useful information is lost.

3.6. Power

For development and evaluation the glove is powered from a regulated 5 V supply over USB-C; each XIAO regulates this to 3.3 V on-board. Power integrity proved to be a practical constraint: the simultaneous radio transmissions of several modules produce brief current spikes, and a marginal supply or wiring path can sag the rail far enough to trigger the ESP32-C3 brown-out detector and reset a module. The mitigations—reducing the Wi-Fi transmit power to a level appropriate for a few-centimetre link, improving the power wiring, and adding local decoupling capacitance—are discussed with the evaluation in Chapter 7.

4. FIRMWARE AND SENSOR SIGNAL PROCESSING

The firmware exists in two roles, sharing a common signal-processing core: the *slave* sketch (thumb, index, ring, pinky) and the *master* sketch (middle finger). Both are written for the Arduino-ESP32 framework.

4.1. Per-Finger Signal Processing

Every module, master or slave, processes its own IMU identically.

4.1.1. Acquisition and Filtering

The MPU6050 is read over I²C at 25 Hz. The on-sensor digital low-pass filter is set to 10 Hz, which removes high-frequency mechanical noise before sampling. A bus-recovery routine bit-bangs the clock line and re-initialises the sensor if a sequence of reads fails, so a transient I²C stall does not permanently disable a finger.

4.1.2. Orientation Estimation

Pitch and roll are estimated by fusing the accelerometer and gyroscope with a two-state Kalman filter [2] per axis, with the angle and the gyro bias as states. When the magnitude of the measured acceleration departs from 1 *g* (indicating linear motion rather than pure gravity), the filter trusts the gyroscope more and gates out the unreliable accelerometer tilt. Yaw, which a six-axis IMU cannot reference absolutely, is obtained by integrating the bias-corrected *z*-axis angular rate with a small dead-band, and is reset to zero after a sustained period of stillness to bound its drift.

4.1.3. Calibration and Bias Tracking

At start-up each module performs a stillness calibration: it averages several hundred samples to establish the zero references for pitch and roll and the initial gyroscope bias, rejecting the attempt if the measured variance shows the hand was moving. During operation, whenever the module detects stillness (low angular rate and acceleration close to 1 *g*), it slowly updates the gyroscope bias with an exponential

moving average. This continuously tracks the bias drift caused by the microcontroller warming the adjacent sensor, which is significant for a low-cost gyroscope and is one reason motion-based features later proved more robust than absolute-orientation features.

4.1.4. Packet Format

Each module produces, per cycle, a packet of nine fields: the three fused angles (pitch, roll, yaw), the three bias-corrected angular rates (g_x, g_y, g_z), and the three accelerations (a_x, a_y, a_z). A slave sends this as a small fixed-size structure tagged with its finger identifier and a sequence number.

4.2. The Slave Sketch

A slave runs a deadline-scheduled 25 Hz loop. Each cycle it reads and fuses its IMU, builds its packet, and transmits it by ESP-NOW *unicast* to the master's address on a fixed Wi-Fi channel. A staggered start-up delay per finger reduces the chance that all four slaves transmit in the same instant. The module also prints a periodic diagnostic line (read success rate, transmit success/failure counts, current angles) over USB for bench debugging.

4.3. The Master Sketch

The master is dual-purpose, selected at compile time:

- **CSV mode** streams every received packet, plus its own, as comma-separated lines over USB at 921600 baud. This feeds the data-collection script (Chapter 5). The high baud rate is necessary: at 115200 baud the five-finger stream saturates the USB serial buffer, the blocking print inside the receive callback backs up, and 15–30% of packets are dropped—a failure that was diagnosed and fixed by raising the baud rate.
- **BLE mode** exposes a GATT service with a notify characteristic. Each 25 Hz cycle the master reads its own IMU, packs the five fingers' nine fields into a 180-byte payload (45 little-endian 32-bit floats), and sends it as a BLE notification to the connected phone. A finger whose data has not arrived recently is sent as zeros so the payload is always well-formed.

4.3.1. Aggregation and Channel Order

The master keeps the latest packet from each finger in a lock-protected table. The 45 channels are emitted in a fixed finger-major order—thumb, index, middle, ring, pinky—each contributing $P, R, Y, g_x, g_y, g_z, a_x, a_y, a_z$. This exact order is mirrored in the data-collection script and in the phone application, so that the model sees identically ordered channels everywhere.

4.3.2. Radio Coexistence and the BLE MTU

The master runs ESP-NOW (Wi-Fi physical layer) reception and BLE transmission on a single radio, which the ESP32-C3 time-slices. At 25 Hz this is comfortable. One non-obvious requirement is the BLE *maximum transmission unit*: the default 23-byte MTU carries only 20 payload bytes per notification, far short of the 180-byte feature vector, so the phone must negotiate a larger MTU (247 bytes requested, 256 granted) immediately after connecting. Without this negotiation the notifications are truncated and silently rejected by the phone—a failure that initially presented as “no data” despite a healthy radio link.

5. DATA ACQUISITION, FEATURE ENGINEERING, AND MODELLING

This chapter describes how gesture data is collected, how it is represented, and how the recognition model is trained and deployed. The feature-representation study in Section 5.3 is the central technical contribution of the project.

5.1. Data Acquisition

Data is collected with the master in CSV mode (Chapter 4) connected to a laptop, driven by a Python recorder. The operator selects a label, the recorder shows a short countdown, and a fixed-length window is captured while the gesture is performed. Each saved sample is a CSV file of 75 rows ($3\text{ s} \times 25\text{ Hz}$) and 45 channels. The recorder forward-fills the most recent packet from each finger onto a uniform time grid and flags samples in which any finger’s data went stale, so that dropped-packet windows can be discarded.

5.2. Signal Window and Channels

A gesture is represented by a 75×45 window: 75 time steps of the 45 channels defined in Chapter 4. This window length and channel order are fixed across data collection, training, and on-device inference. Matching them exactly—*train/inference parity*—is essential; a mismatch in window length, sampling rate, or channel order silently degrades or breaks live accuracy even when offline metrics look perfect.

5.3. Feature Representation

The most important design question proved to be *what* the model sees, not which classifier is used.

5.3.1. The Problem with Absolute Orientation for Static Handshapes

A natural first representation is the absolute fused orientation of each finger. This failed in live use for static fingerspelling in two distinct ways. First, the absolute pitch/roll of each finger is measured relative to the start-up calibration pose, which a

user cannot reproduce exactly between sessions; a few degrees of difference in how the hand is held shifts every channel and causes confident misclassification. Second, even with perfect calibration, letters that differ only by small thumb placement (e.g. **A** vs **T**) lie within the angular noise floor of the sensors and cannot be separated.

5.3.2. Relative (Orientation-Invariant) Features

To address the calibration sensitivity, an orientation-invariant transform was introduced. For the orientation-bearing channels (P, R, Y, a_x, a_y, a_z), the per-time-step mean across the five fingers is subtracted, leaving only the *inter-finger* differences; the angular-rate channels are left unchanged. Because a global rotation of the hand (or a calibration offset) adds the same amount to all five fingers, subtracting their mean cancels it, while the relative finger configuration—which defines the handshape—is preserved. This made static-letter features invariant to global hand tilt and to the calibration pose.

Relative features removed the calibration sensitivity but, as expected, could not overcome the sensor-resolution limit: the thumb-cluster letters remained confusable, because the information that separates them was never captured by the sensors in the first place.

5.3.3. Dynamic Gestures with Absolute Features

The resolution of both problems was to change the gestures rather than only the features. Each letter (except the inherently dynamic **J** and **Z**) was re-recorded as a short, distinctive *motion* that engages specific fingers—for example a thumb fold, an index-and-thumb tap, or a thumb-and-pinky meeting—so that the discriminative information lives in the angular-rate (g) channels. The angular rate is a measurement of motion and is inherently independent of the start-up calibration, so *absolute* features (no mean-centering) can be used. This dynamic-gesture, absolute-feature formulation is used for both the letter and the word models. A small number of practical rules emerged: the motion must fill the whole capture window (a gesture with a long still tail collapses toward a low-motion “attractor” class), and within a confusable group the motions must differ clearly in axis, direction, or repetition count.

The same logic was confirmed for word signs. Word pairs distinguished only by global hand orientation or translation (e.g. **YOU** vs **ME**) collapsed under mean-centering—which deliberately discards exactly that information—and were correctly separated once absolute features were used.

5.4. Augmentation and Dataset Splitting

Recording every gesture across many sessions is time-consuming, so synthetic augmentation is used to improve robustness to session-to-session variation. For each training sample, several copies are generated with small random per-finger angle offsets, small per-finger gyroscope-bias offsets, and per-reading sensor noise—simulating realistic calibration and thermal variance without re-recording. Critically, the train/test split is performed *before* augmentation and on the original samples only, so that no augmented copy of a test sample can appear in training. This leakage-safe procedure keeps the held-out test set an honest measure of generalisation.

5.5. Network Architecture

The classifier is a compact one-dimensional convolutional neural network [4]. The flattened input is reshaped to 75×45 and passed through two convolutional blocks—each a 1-D convolution, batch normalisation, max-pooling, and dropout (16 then 32, or 32 then 64, filters, kernel size 3)—followed by a flatten, a dense layer, dropout, and a softmax output over the classes. The architecture was authored in Keras through Edge Impulse’s expert mode [7]. Training uses categorical cross-entropy with the Adam optimiser, a 20% validation split, and standard-scaler normalisation of the inputs, whose per-channel means and standard deviations are exported so the phone can reproduce the identical normalisation at inference time.

5.6. Quantisation and Deployment

The trained model is exported as an 8-bit-integer (int8) quantised TensorFlow Lite file [6]. Quantisation reduces the model size roughly four-fold and speeds up inference, and on this data it incurred no measurable accuracy loss. The deployed model occupies about 137 KB of flash and 22.7 KB of RAM and runs in approximately 1 ms per inference (Chapter 7). Because the model’s input and output tensors are both int8, the application must quantise the normalised input and dequantise the output using the scale and zero-point published in the model—handled in the mobile application (Chapter 6).

6. MOBILE APPLICATION

The mobile application, built with Flutter for cross-platform deployment, performs three roles: it manages the BLE link to the glove, it runs the on-device neural network, and it presents recognised gestures as text and speech.

6.1. Architecture

The application is organised into services with clear responsibilities: a BLE service that handles scanning, connection, and packet parsing; an inference service that buffers sensor frames and runs the model; a text-to-speech service; and a preferences service that holds the per-model configuration. The main screen is a single capture-and-display view; a segmented control switches between the *Letters* and *Words* models.

6.2. Bluetooth Link and Packet Parsing

On connection the BLE service requests the appropriate runtime permissions, scans for the glove by address (falling back to its advertised name), connects, and crucially negotiates a larger MTU before subscribing to the notify characteristic (Chapter 4). Each 180-byte notification is parsed into 45 little-endian floats forming one sensor frame, which is pushed into a fixed-size rolling buffer. Frames are rejected if their length is wrong or any value is non-finite. The Android manifest declares the modern `BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` permissions; an early bug, in which the app required a location permission that the manifest no longer declared on recent Android, was traced to this and fixed.

6.3. Gated Capture

Rather than classifying continuously, the application uses *gated capture*, mirroring the data-collection procedure exactly. The user presses a **Listen** button; the screen shows a brief countdown so the signer can prepare, then a clear “sign now” cue while a fresh window of sensor frames is collected; the model is then run once on that window. This guarantees that the window seen at inference matches the window used during training, and it makes the device’s behaviour predictable and demonstrable. It also matches how fingerspelling is naturally performed: one deliberate gesture at a time.

6.4. On-Device Inference Pipeline

When a window is captured, the inference service applies, in order: (i) the per-model feature transform (the relative transform for a relative model, or none for an absolute model), (ii) standard-scaler normalisation using the model’s exported statistics, (iii) quantisation of the normalised values to int8 using the input tensor’s scale and zero-point, (iv) execution of the TensorFlow Lite model, and (v) dequantisation of the int8 output back to class probabilities, followed by an arg-max. Each of these steps must agree exactly with the training pipeline; the feature transform and the normalisation statistics are therefore selected *per model* (Section 6.5).

6.5. Per-Model Configuration

The Letters and Words models have different class sets, different normalisation statistics, and—in principle—could use different feature transforms. The application therefore swaps the model file, the label list, the normalisation statistics, and the transform flag together whenever the user changes mode. A subtle bug class this avoids is silently using one model’s normalisation with another model’s weights; the normaliser is reset on each load so a failed swap can never leave stale statistics in place.

6.6. Output

In Words mode, a recognised sign is spoken immediately through text-to-speech. In Letters mode, recognised letters accumulate into a text buffer so the user can fingerspell a word; a **Speak** button reads the assembled word aloud and a **Clear** button empties the buffer to start again. A confidence threshold suppresses low-confidence predictions so that an uncertain capture asks the user to try again rather than committing a wrong letter.

Figure 6.1 shows the application in use. In Words mode (left) the recognised sign is shown in large type with a confidence bar beneath it and is spoken at once; multi-word signs such as HOW ARE YOU are displayed and spoken as spaced phrases rather than the run-together tokens the model emits. In Letters mode (centre) each accepted letter is appended to the buffer shown at the top, and the **Speak** and **Clear** controls let the user read back or reset the assembled word. The Settings screen (right) exposes the confidence threshold, the text-to-speech voice, and the Letters-mode auto-clear interval. The same confidence threshold governs both modes, so the interface stays quiet rather than guessing when a capture is ambiguous.

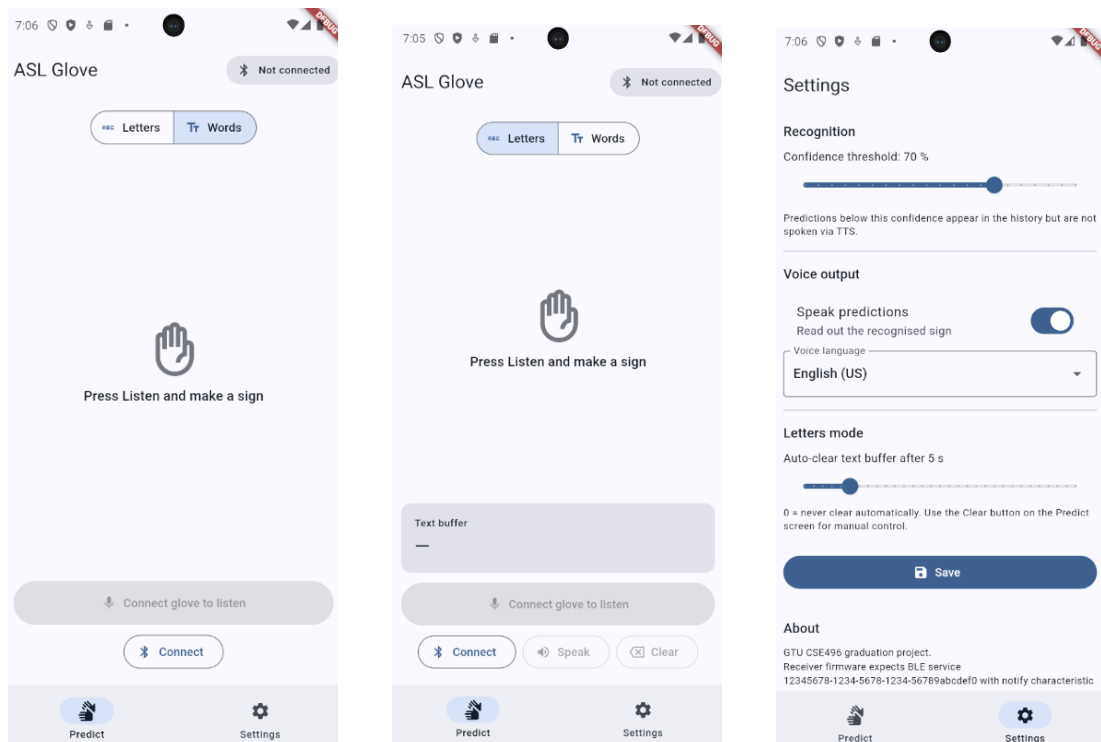


Figure 6.1: The mobile application. Left: Words mode, which speaks a recognised sign immediately. Centre: Letters mode, which accumulates fingerspelled letters into a buffer with **Speak** and **Clear** controls. Right: the Settings screen, exposing the confidence threshold, text-to-speech voice, and the Letters-mode auto-clear interval.

7. RESULTS AND EVALUATION

This chapter reports the measured behaviour of the system across four dimensions: the wireless link, on-device model performance, recognition accuracy, and power/thermal robustness. Where a result has an important limitation, it is stated plainly.

7.1. Wireless Link Reliability

With the master in BLE mode and all four slaves powered, the master’s periodic diagnostics report the number of packets received from each finger over each five-second window. In a healthy run every finger reports approximately 125 packets per five seconds—exactly the expected $25 \text{ Hz} \times 5 \text{ s}$ —with the per-finger loss counters at zero (Table 7.1). The BLE link, once the MTU is negotiated to 256 bytes, delivers the full 180-byte feature vector per notification at the same 25 Hz. The architecture change from a wrist hub with external antennas to nail-mounted modules on the on-chip antennas was decisive here: at the few-centimetre finger-to-finger distance the link is no longer pose-dependent, and packet loss in the closed-fist configuration—previously the worst case—is negligible.

Table 7.1: Representative five-second link diagnostics from the master (BLE connected).

Finger	Packets received	Lost
Thumb	125	0
Index	125	0
Middle (own)	125	0
Ring	125	0
Pinky	125	0

7.2. On-Device Model Performance

The deployed int8 model is small and fast (Table 7.2). At roughly 1 ms per inference against a 40 ms (25 Hz) frame budget, inference latency is negligible, and the memory footprint is small enough for any modern phone. The gated-capture design means inference runs once per user request rather than continuously, so the practical

compute and energy cost is lower still.

Table 7.2: On-device characteristics of the quantised model (Edge Impulse estimate, Raspberry Pi 4 class target).

Metric	Value
Inference time	≈ 1 ms
Peak RAM	22.7 KB
Flash (model)	137 KB
Quantisation	int8 (input and output)

7.3. Recognition Accuracy

Several models were trained over the course of the project as the feature representation evolved. Table 7.3 summarises the validation-set accuracy of the main variants, as reported by Edge Impulse on the final training run of each. The held-out split is taken *before* augmentation, so augmented copies of validation samples never appear in training and the figures are not inflated by that form of leakage.

Table 7.3: Edge Impulse validation-set accuracy of the main model variants. All figures are within-session (the data is recorded in a single session and split internally); they measure separability under fixed calibration, not cross-session generalisation—see the caveat below.

Model	Classes	Features	Accuracy
Smoke set	5	raw / static	high (pipeline validation)
Letters v2	26	relative, static	$\approx 100\%$
Words (final)	19	absolute, dynamic	100.0%
Letters v3 (final)	26	absolute, dynamic	100.0%

The accuracy figures alone do not show *which* classes were confused, which is where the feature representation made its difference. Figure 7.1 shows the validation-set confusion matrices exported from Edge Impulse for the final letters and words models. Both are fully diagonal: every class is separated cleanly with no residual within-session confusions. This is exactly what the move to dynamic gestures with absolute features bought, in contrast to the systematic off-diagonal errors (e.g. T $\rightarrow$ A, YOU $\leftrightarrow$ ME) that the earlier static and relative representations produced. The meaningful residual errors are no longer within-session—they appear only in live, cross-session use, which is discussed next.



Figure 7.1: Validation-set confusion matrices from Edge Impulse. Top: the 26-letter model. Bottom: the 19-word model. Both are fully diagonal—every class is recognised with no within-session confusion—so the residual errors of interest are the cross-session ones discussed in the live findings below, not anything visible here.

7.3.1. The Single-Session Caveat

The validation accuracies in Table 7.3 reach 100%, but they are *within-session*: each validation split shares the same calibration, mounting, and thermal state as its training set. This is the easiest possible test and must not be read as the system’s accuracy for a fresh user session. The honest measure of generalisation is cross-session—training on one or more sessions and testing on a different day with a fresh calibration—and the live testing described next is the practical proxy for it.

7.3.2. Live Findings and the Role of Feature Representation

Live testing was the experiment that drove the project’s main finding. The static, absolute-feature letter model achieved high offline accuracy yet failed live, confidently confusing letters because the calibration pose could not be reproduced. Introducing the relative (orientation-invariant) transform removed this calibration sensitivity, but the thumb-cluster letters (**A/S/T/M/N**) remained confused live, confirming that their separation is limited by sensor resolution rather than by the feature transform. For the word model, the relative transform introduced a different failure: signs distinguished only by global hand orientation or translation (notably **YOU** vs **ME**) collapsed, because mean-centering deliberately discards exactly that information; switching the word model to absolute features separated them again.

These observations converge on a single conclusion: for inertial sign recognition on low-cost sensors, *dynamic gestures with absolute features* are the robust choice, because the discriminative information then resides in the angular-rate channels, which are inherently independent of the start-up calibration. Re-recording the alphabet as distinctive per-letter motions and training on absolute features is the embodiment of this conclusion. A secondary but practical finding is that a dynamic gesture must fill the capture window: a sign with a long still tail drifts toward whichever class has the least motion, which then acts as an attractor for ambiguous captures.

7.4. Power and Thermal Behaviour

The peak instantaneous current of the whole glove is dominated by simultaneous radio transmissions. With the Wi-Fi transmit power at its maximum, a slave’s transmit burst draws on the order of 300 mA; in the pathological case of all four slaves transmitting in the same instant the system peak approaches 1.5 A, although the phase-staggered transmissions make full alignment momentary. The average draw is far lower, on the order of a few hundred milliamps.

This peak is the source of the most disruptive failure observed: a marginal supply or wiring path sags the 3.3 V rail far enough during a transmit burst to trip the ESP32-C3 brown-out detector and reset a module, which in the logs appears as a repeated reboot and (on the affected board) erratic sensor reads. Because the transmit current is the trigger, the primary mitigation is to reduce the Wi-Fi transmit power: the finger-to-finger link spans only a few centimetres and has tens of decibels of margin, so a much lower transmit power both removes the current spike and reduces self-heating, with no loss of link reliability. Improving the power wiring and adding local decoupling capacitance at each module are complementary hardware mitigations. Reducing the transmit power changes only the radio, not the sensor data, so it requires no re-collection of data or re-training.

Separately, the proximity of the microcontroller to the IMU within each nail module warms the gyroscope over a session, which shifts its zero-rate bias. This is handled in firmware by the continuous still-state bias tracking (Chapter 4) and is a further reason the motion-based, calibration-independent features were preferred.

8. CONCLUSION AND FUTURE WORK

8.1. Summary

This project set out to translate American Sign Language gestures into speech using only a wearable inertial glove, with the entire recognition pipeline running on commodity hardware and a phone. The resulting system places a microcontroller-and-IMU module on each fingernail, streams four fingers’ data over ESP-NOW to a fifth “master” finger, and bridges to a mobile phone over Bluetooth Low Energy, where a quantised one-dimensional convolutional neural network classifies a captured window and speaks the result. The wireless link delivers all five finger streams at 25 Hz with negligible loss, and the deployed model runs in about one millisecond using tens of kilobytes of memory.

Beyond the working system, the project’s most valuable outcome is a clear, empirically grounded understanding of *feature representation* for inertial sign recognition. Through successive live failures and fixes, the work established that absolute-orientation features for static handshapes are fragile against imperfect re-calibration and limited by sensor resolution; that an orientation-invariant relative transform fixes the calibration problem but not the resolution problem, and harms dynamic signs that depend on global motion; and that the robust formulation is to perform gestures as distinctive *motions* and classify them on *absolute* features, placing the decisive information in the calibration-independent angular-rate channels.

8.2. Limitations

Several limitations are acknowledged honestly. First, the reported accuracies are within-session; rigorous cross-session and cross-user evaluation remains to be done, and is expected to lower the figures. Second, the device instruments a single hand and recognises segmented, one-gesture-at-a-time input, not continuous or two-handed signing. Third, fingerspelling letters that differ only by fine thumb placement are at the limit of what six-axis IMUs can resolve statically; the project mitigates this by encoding letters as distinct motions, but this departs from the canonical static handshapes. Fourth, power integrity under simultaneous radio transmissions is a real constraint that requires attention to the supply and wiring. Finally, recognition operates at the level of single letters and words without higher-level language modelling.

8.3. Future Work

- **Cross-session and multi-user data.** Collecting several sessions on different days, and data from multiple wearers, would both improve robustness and provide an honest generalisation measure to replace the within-session figures.
- **Word-level language modelling.** A dictionary- or trie-based decoder over the letter stream would resolve residual letter confusions at the word level, since context disambiguates letters that are individually ambiguous—mirroring how human readers parse fingerspelling.
- **Power hardening and battery operation.** Fixing the supply and wiring, adding decoupling, lowering the transmit power, and adding a listen-triggered sleep mode would enable untethered battery operation and reduce heat, allowing longer continuous use.
- **Larger and two-handed vocabulary.** Extending to a larger word set and, with a second instrumented hand, to two-handed signs would broaden practical usefulness.
- **Continuous recognition.** Moving from gated, segmented capture to continuous gesture spotting would make the device feel more natural, at the cost of a harder segmentation and modelling problem.

8.4. Concluding Remarks

The project delivers a complete, self-contained sign-language translation glove and, just as importantly, a documented account of why it is built the way it is. The system evolved through measured failures—antenna coupling, USB throughput, BLE MTU, calibration drift, sensor resolution, and power integrity—each of which was diagnosed and resolved with an understood engineering trade-off rather than a guess. That process, and the feature-representation conclusion it produced, is the work’s main contribution and the strongest foundation for the future work above.

BIBLIOGRAPHY

- [1] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, Document Number: PS-MPU-6000A-00, InvenSense Inc., 2013.
- [2] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [3] Espressif Systems. “ESP-NOW User Guide,” Accessed: Jun. 13, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp-now_user_guide_en.pdf.
- [4] C. A. Ronao and S.-B. Cho, “Human activity recognition with smartphone sensors using deep learning neural networks,” *Expert Systems with Applications*, vol. 59, pp. 235–244, 2016.
- [5] M. Zeng et al., “Convolutional neural networks for human activity recognition using mobile sensors,” in *6th International Conference on Mobile Computing, Applications and Services (MobiCASE)*, 2014, pp. 197–205.
- [6] R. David et al., “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.
- [7] S. Hymel et al., “Edge impulse: An MLOps platform for tiny machine learning,” *Proceedings of Machine Learning and Systems (MLSys)*, 2023.
- [8] Seeed Studio. “Seeed Studio XIAO ESP32-C3 Wiki,” Accessed: Jun. 13, 2026. [Online]. Available: https://wiki.seeedstudio.com/XIAO_ESP32C3_Getting_Started/.

APPENDICES

Appendix 1: Channel Layout of the Feature Vector

Each 25 Hz sample is a 45-channel vector in finger-major order. For each finger in the order thumb, index, middle, ring, pinky, the nine fields are: pitch (P), roll (R), yaw (Y), angular rate about $x/y/z$ (g_x, g_y, g_z), and linear acceleration along $x/y/z$ (a_x, a_y, a_z). This ordering is identical in the data-collection script, the master firmware payload, the normalisation statistics, and the mobile application’s parser.

Appendix 2: Vocabulary

Letters: the 26 letters A–Z. Most are encoded as short, distinctive finger motions; J and Z are the inherently dynamic ASL letters.

Words (19 one-handed signs): DRINK, EAT, FINE, HELLO, HOWAREYOU, ILOVEYOU, ME, NAME, NO, PLEASE, SORRY, STOP, THANKYOU, WATER, WHAT, WHERE, YES, YOU, YOUR. The compact multi-word tokens (e.g. HOWAREYOU, ILOVEYOU) are expanded to spaced phrases (“HOW ARE YOU”, “I LOVE YOU”) for display and speech in the application.

Appendix 3: Key System Parameters

Parameter	Value
Fingers / channels	5 / 45
Sampling rate	25 Hz
Capture window	3 s (75 time steps)
ESP-NOW channel	11
BLE MTU (negotiated)	256 bytes
BLE payload	180 bytes (45 float32)
Master CSV baud rate	921600
Model quantisation	int8 (input and output)

Appendix 4: Source Code

The firmware, data-processing scripts, and mobile application are organised in the project repository under `firmware/`, `scripts/`, and `flutter_app/` respectively.